



TD 06 - Vérifier avec Sonarqube

SonarQube est une plateforme d'analyse de la qualité du code qui permet d'évaluer, surveiller et améliorer la qualité du code source d'un projet. Il prend en charge plusieurs langages de programmation dont Java et Python. Ce TD vous apprendra à utiliser les rapports générés par SonarQube pour détecter et corriger les bugs, vulnérabilités et mauvaises pratiques.

Objectifs

À l'issue de ce TD, vous serez capable d' :

- Installer et configurer SonarQube via Docker pour analyser des projets dans différents langages.
- Analyser un projet Java avec SonarQube pour identifier des erreurs, une complexité excessive, des vulnérabilités et une couverture de tests insuffisante.
- Analyser un projet Python avec SonarQube et interpréter les résultats d'analyse similaires à ceux du projet Java.

PRÉ-REQUIS

1. Connaissance de base en Java, Python, Docker et des commandes shell.
2. Un environnement de travail prêt avec Java (JDK 17 minimum), Python 3.8+ et un IDE.

Installation de SonarQube avec Docker

Création d'un réseau Docker

SonarQube se compose de deux conteneurs distincts :

- Le serveur SonarQube
- Le scanner SonarQube

Pour permettre leur communication, il est nécessaire de créer un réseau Docker dédié. Exécutez la commande suivante pour créer ce réseau :

```
docker network create sonar-network
```

Installation et démarrage du serveur SonarQube

Le serveur SonarQube est le cœur du système. Il est responsable du stockage, de l'analyse et de l'affichage des résultats des analyses de code. Il comprend plusieurs composants :

- Base de données : Stocke les résultats des analyses, les configurations des projets et les paramètres de qualité.
- Moteur d'analyse : Traite les rapports envoyés par les scanners et applique les règles de qualité du code.
- Interface web : Permet aux développeurs et aux équipes de visualiser les rapports, d'explorer les problèmes détectés et de suivre la qualité du code.

Exécutez la commande suivante pour lancer un conteneur SonarQube sur le réseau Docker créé précédemment :

```
docker run -d --name sonarqube --network sonar-network -p 9000:9000  
sonarqube:lts-community
```

Cette commande démarre SonarQube en mode détaché (`-d`), le connecte au réseau `sonar-network` et l'expose sur le port `9000`.

Les informations sur cette image sont disponibles sur [Docker Hub](#).

ATTENTION

Consultez les logs du conteneur SonarQube pour savoir si il a démarré avec succès.

Connexion à l'interface SonarQube

Une fois SonarQube démarré, ouvrez un navigateur et accédez à l'URL <http://localhost:9000>.

Utilisez les identifiants par défaut pour vous connecter :

- *Nom d'utilisateur* : admin

- *Mot de passe* : admin

Création d'un token d'authentification

Un token d'authentification est nécessaire pour permettre au scanner SonarQube d'envoyer les résultats d'analyse au serveur SonarQube, sans avoir à saisir manuellement des identifiants.

Une fois connecté au serveur SonarQube :

- Allez dans *Mon compte* (en haut à droite).
- Sélectionnez l'onglet *Security*.
- Remplissez le formulaire de génération d'un token :
 - *Name* : premier-token
 - *Type* : Global Analysis Token
 - *Expires in* : 30 jours
- Cliquez sur *Generate*.
- Copiez le token généré, car il ne sera plus affiché.

Installation de Sonar Scanner CLI

Le scanner SonarQube est l'outil qui analyse le code source avant d'envoyer les résultats au serveur. Il fonctionne de la manière suivante :

- Il analyse le code source selon un ensemble de règles prédéfinies.
- Il génère un rapport contenant les erreurs, les vulnérabilités, la complexité du code, la dette technique,...
- Il envoie ce rapport au serveur SonarQube pour traitement et affichage dans l'interface web.

Téléchargez et installez l'outil Sonar Scanner CLI pour analyser votre code via l'image dédiée :

```
docker pull sonarsource/sonar-scanner-cli
```

Analyse d'un projet java

Dans cet exercice, vous allez découvrir comment analyser la qualité du code d'un projet Java en utilisant SonarQube. L'objectif est de configurer un projet, d'exécuter une analyse avec Sonar

Scanner et de consulter les résultats sur le serveur SonarQube. Ce premier exemple est repris du dépôt [sonar-training-exemples](#).

Commencez par créer le projet java `quality-demo` et ajoutez-y la classe `HelloWorld`.

HelloWorld.java

```
1 public class HelloWorld {
2
3     public static void main(String[] args) {
4
5         System.out.println("Hello World !");
6         System.out.println("Hello World !");
7         System.out.println("Hello World !");
8         System.out.println("Hello World !");
9         System.out.println("Hello World !");
10        System.out.println("Hello World !");
11        System.out.println("Hello World !");
12        System.out.println("Hello World !");
13        System.out.println("Hello World !");
14        System.out.println("Hello World !");
15        System.out.println("Hello World !");
16        System.out.println("Hello World !");
17        System.out.println("Hello World !");
18        System.out.println("Hello World !");
19        System.out.println("Hello World !");
20        System.out.println("Hello World !");
21    }
22
23 }
```

Première analyse : un succès

Créez à la racine du projet le fichier `sonar-project.properties` ci-dessous. Ce fichier de configuration est utilisé par Sonar Scanner pour définir les paramètres d'analyse d'un projet. Il spécifie des informations essentielles telles que le nom du projet, la version, les chemins des fichiers source et des tests.

```
# Identifiant du projet.
# Doit être unique dans l'instance SonarQube.
# Obligatoire
sonar.projectKey=quality-demo-java
```

```
# Nom du projet tel qu'il apparaîtra dans l'interface web.
# Obligatoire
sonar.projectName=Application de demonstration en Java

# Permet de suivre l'évolution de la qualité entre différentes versions
# Facultatif
sonar.projectVersion=1.0

# Répertoire de base du projet
# Par défaut SonarScanner utilise le dossier où il est exécuté.
# Facultatif
sonar.projectBaseDir=.

# Spécifie où se trouvent les fichiers source à analyser.
# Obligatoire
sonar.sources=src/main/java

# Répertoire contenant les fichiers de bytecode compilés (.class)
# Obligatoire pour Java. SonarQube a besoin des fichiers
# compilés pour effectuer certaines analyses (comme la détection
# des problèmes de type ou de complexité cyclomatique).
sonar.java.binaries=target/classes
```

💡 PROPRIÉTÉ SPÉCIFIQUE À UN LANGUAGE

Il est important de noter que la propriété `sonar.java.binaries` est spécifiquement dédiée à la configuration des fichiers binaires **Java** (comme les fichiers `.class`), tandis que `sonar.sources` s'applique à tous les langages du projet, permettant de spécifier les répertoires contenant le code source de l'ensemble du projet, **quel que soit le langage** utilisé.

Dans le dossier racine de votre projet, exécutez la commande suivante, en y ajoutant votre token, pour démarrer l'analyse :

```
docker run --rm --network sonar-network -v "$(pwd):/usr/src" -e
SONAR_HOST_URL="http://sonarqube:9000" -e SONAR_TOKEN="VOTRE_TOKEN_ICI"
sonarsource/sonar-scanner-cli
```

💡 DOCKER RUN

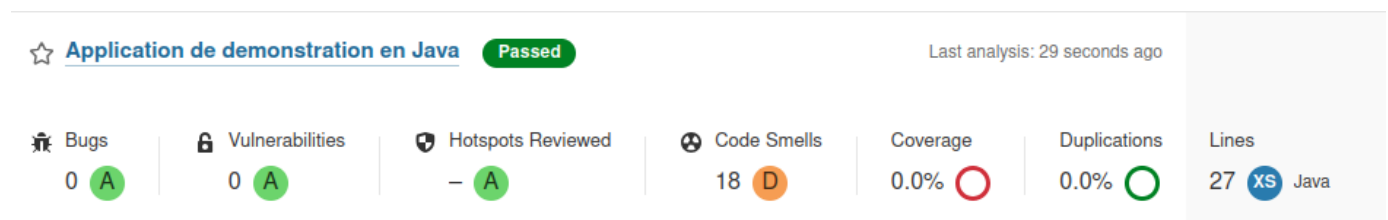
Testez votre mémorisation des paramètres des commandes Docker en essayant d'expliquer les paramètres :

- `--rm`

- `-v "$(pwd):/usr/src"`
- `-e SONAR_HOST_URL="http://sonarqube:9000"`

N'hésitez pas à [consulter la documentation](#) en cas de doute et à en discuter avec votre enseignant·e.

Une fois l'analyse terminée, consultez le résultat sur le serveur <http://localhost:9000/projects>. Vous devriez obtenir un résultat similaire à celui présenté ci-dessous.



! CATÉGORIES D'ANALYSE

Lorsqu'un projet est analysé dans SonarQube, plusieurs catégories permettent d'évaluer la qualité du code, la sécurité, et la maintenabilité. Voici un aperçu détaillé de ces catégories :

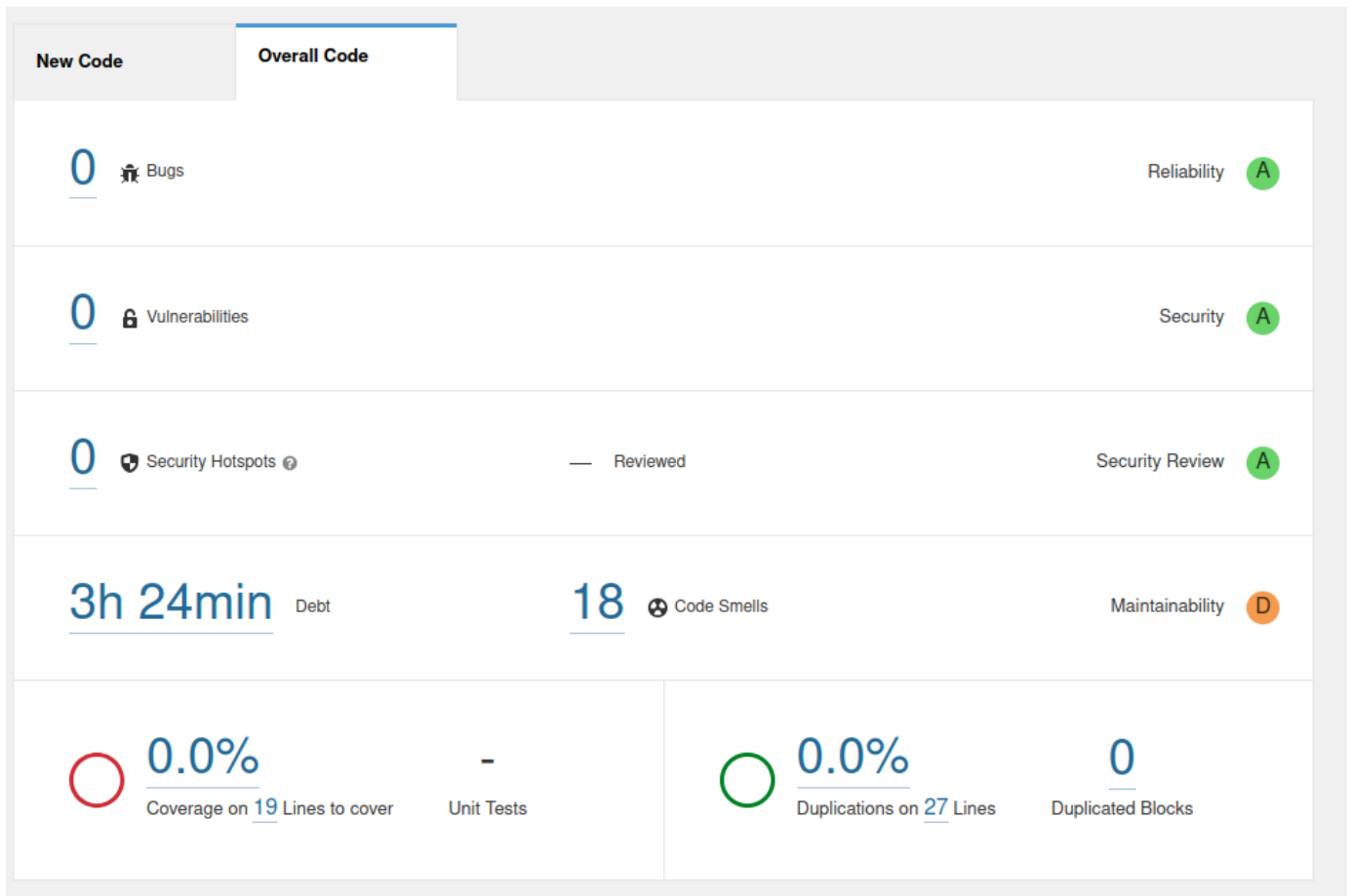
- **Bugs** : Ce sont des erreurs dans le code qui peuvent provoquer des comportements inattendus, des plantages ou des résultats incorrects. Par exemple :
 - Accès à une variable null pouvant entraîner une `NullPointerException`.
 - Boucle infinie causant un blocage du programme.
 - Erreur de logique (exemple : une condition if toujours fausse).
- **Vulnerabilities** : Cette catégorie regroupe les failles de sécurité pouvant être exploitées par un attaquant. Par exemple :
 - Injection SQL (SQL Injection).
 - Faille XSS (Cross-Site Scripting).
 - Stockage de mots de passe en clair dans le code.
- **Hotspots Reviewed** : Ce sont des zones du code sensibles sur le plan de la sécurité, mais qui ne sont pas nécessairement des vulnérabilités. Elles nécessitent une revue manuelle pour évaluer si un risque existe.
- **Code Smells** : Indique les problèmes de maintenabilité qui rendent le code plus difficile à comprendre ou à modifier. Par exemple :
 - Fonctions trop longues ou trop complexes.
 - Duplication excessive de code.
 - Mauvaise utilisation des conventions de codage.

- **Coverage** : Mesure le pourcentage de code couvert par des tests unitaires. Une faible couverture indique que certaines parties du code ne sont pas testées.
 - Une couverture de 80% signifie que 20% du code n'a pas été testé.
- **Duplications** : Analyse la répétition excessive de blocs de code dans le projet.
 - Copier/coller d'un même bloc de logique dans plusieurs fichiers.
 - Redondance de méthodes avec des fonctionnalités identiques.
- **Lines** : Indique le nombre total de lignes de code dans le projet. Cette métrique permet d'avoir une idée de la taille et de la complexité du projet.
 - Un projet avec 50 000 lignes de code est généralement plus difficile à maintenir qu'un projet de 5 000 lignes.

Dans SonarQube, chaque catégorie est accompagnée d'une note de gravité représentée par des lettres (A, B, C, D, E) pour indiquer l'importance des problèmes détectés dans le code. Ces lettres permettent de prioriser les actions à prendre pour corriger les problèmes.

- **A** : Problème très faible ou informatif, sans impact notable, peut être ignoré ou corrigé plus tard.
- **B** : Problème faible, a peu d'impact, à résoudre à moyen terme.
- **C** : Problème mineur, moins critique, à corriger mais pas urgent.
- **D** : Problème majeur, important mais non bloquant, à résoudre dès que possible.
- **E** : Problème critique, nécessite une attention immédiate et doit être corrigé en priorité.

Cliquez sur le nom du projet et accédez au tableau de bord du projet. Vous devriez obtenir un résultat similaire à celui présenté ci-dessous.



💡 DETTE TECHNIQUE

La dette technique fait référence au compromis entre la rapidité de développement à court terme et la qualité du code à long terme. Elle se produit lorsqu'un développeur choisit une solution plus rapide mais moins idéale ou propre, dans le but de gagner du temps. À mesure que cette dette s'accumule, le code devient de plus en plus difficile à maintenir, à tester et à modifier, nécessitant un investissement supplémentaire pour "rembourser" la dette en réécrivant ou en améliorant le code.

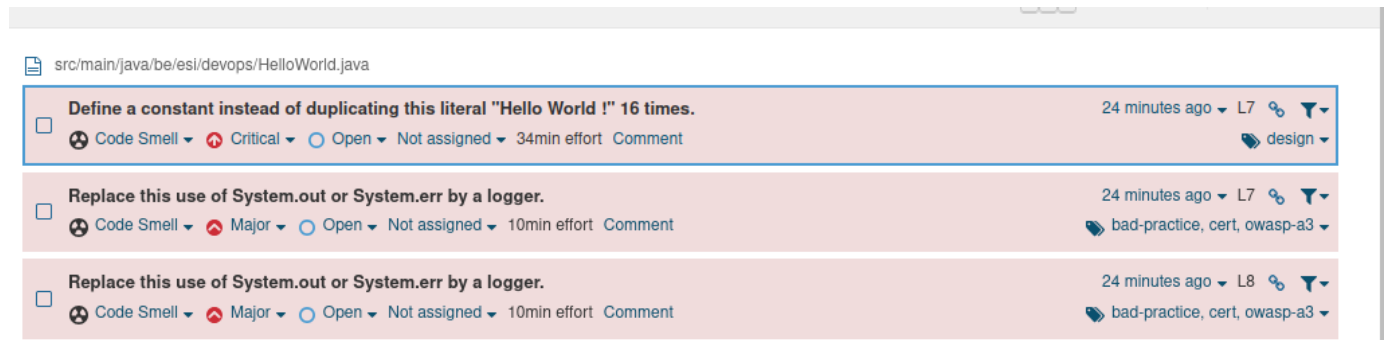
Le nombre d'heures affiché est une estimation qui donne aux développeurs une idée de l'ampleur du travail nécessaire pour améliorer la qualité du code, réduire la dette technique et ainsi améliorer la maintenabilité du projet.

L'analyse du projet est divisée en deux onglets :

- **New Code** met l'accent sur la qualité du code récemment ajouté.
- **Overall Code** offre une vue d'ensemble, incluant à la fois l'ancien et le nouveau code.

Grâce à ces onglets, l'équipe de développement peut suivre la qualité du code en cours tout en gérant progressivement la dette technique.

Consultez la liste des problèmes détectés en cliquant sur le bouton **Code Smells**. Vous devriez obtenir un résultat similaire à celui présenté ci-dessous.



The screenshot shows a list of three issues detected in the file `src/main/java/be/esi/devops/HelloWorld.java`. The first issue is "Define a constant instead of duplicating this literal 'Hello World !' 16 times." with a severity of "Code Smell", a criticality of "Critical", and a status of "Open". The second and third issues are "Replace this use of System.out or System.err by a logger." with a severity of "Code Smell", a criticality of "Major", and a status of "Open". All issues have a "34min effort" and a "Comment" link.

SonarQube détecte clairement un problème de répétition de code, indiquant qu'il est préférable de définir une constante plutôt que de dupliquer 16 fois la chaîne de caractères "Hello World !".

De plus, chaque utilisation de `System.out.println()` est signalée comme un souci. SonarQube recommande d'utiliser un logger à la place, afin d'améliorer la gestion des messages et d'offrir un meilleur contrôle sur les logs de l'application.

En cliquant sur un des problèmes détectés, l'onglet "Why is this an issue ?" fournit une explication détaillée sur la raison de cette recommandation.



The screenshot shows the details of the issue "Define a constant instead of duplicating this literal 'Hello World !' 16 times." The issue is categorized as "Code Smell" and "Critical". The "Where is the issue?" tab is selected, showing the code snippet where the issue occurs: `System.out.println("Hello World !");`. The "Why is this an issue?" tab is also visible.

Modifiez la classe `HelloWorld` pour corriger la duplication signalée

HelloWorld.java

```
1 public class HelloWorld {
2
3     public static void main(String[] args) {
4         System.out.println("Hello World !");
5     }
6 }
```

```
7 }
```

Lancez une nouvelle analyse via la commande :

```
docker run --rm --network sonar-network -v "$(pwd):/usr/src" -e  
SONAR_HOST_URL="http://sonarqube:9000" -e SONAR_TOKEN="VOTRE_TOKEN_ICI"  
sonarsource/sonar-scanner-cli
```

En consultant le rapport sur le serveur, vous constaterez que le nouveau code ne présente plus aucune erreur. Toutefois, l'onglet **Overall Code** signale toujours l'utilisation de l'instruction `System.out`.

Seconde analyse : un échec

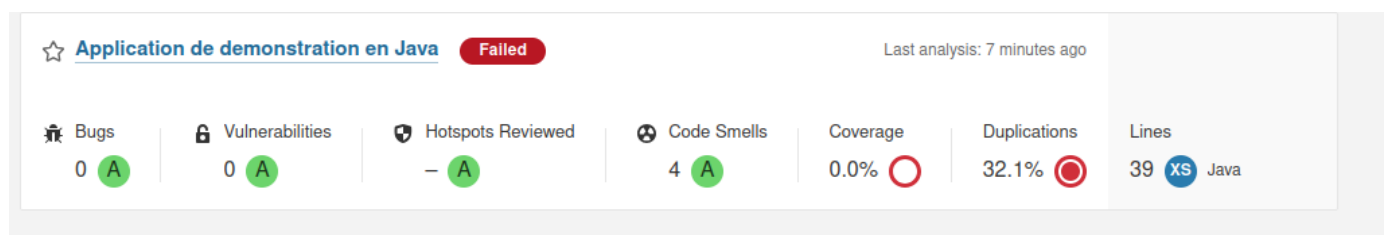
Ajoutez la classe `ComplexMethod` au projet `quality-demo` afin d'explorer un nouveau type d'erreur. La méthode `intToEnglishValue` retourne le mot anglais correspondant au chiffre donné en paramètre.

ComplexMethod.java

```
1 public class ComplexMethod {  
2  
3     public String intToEnglishValue(int i) {  
4         /*         if (i == 0) {  
5             int k = 10 / i;  
6         }  
7     */  
8         if (i == 1) {  
9             return "One";  
10        }  
11        if (i == 2) {  
12            return "Two";  
13        }  
14        if (i == 3) {  
15            return "Three";  
16        }  
17        if (i == 4) {  
18            return "Four";  
19        }  
20        if (i == 5) {  
21            return "Five";  
22        }  
23    }  
24 }
```

```
23         if (i == 6) {
24             return "Six";
25         }
26         if (i > 6) {
27             throw new RuntimeException();
28         }
29         return null;
30     }
31
32 }
```

Relancez une analyse et consultez le rapport.



Votre projet ne respecte pas les critères de qualité attendus, entraînant l'échec de l'analyse. Corrigez les problèmes détectés, puis effectuez une nouvelle analyse. Par exemple la méthode peut retourner la valeur de la case `i` du tableau `String[] values = {"", "One", "Two", "Three", "Four", "Five", "Six"};`.

! QUALITY GATE

La Quality Gate dans SonarQube est un ensemble de critères de qualité définis pour déterminer si un projet respecte les standards exigés avant d'être validé.

Exemple d'un Quality Gate par défaut :

- Aucun bug ou vulnérabilité critique
- Couverture des tests $\geq 80\%$
- Duplication du code $\leq 3\%$

Les Quality Gates peuvent être personnalisés pour s'adapter aux exigences spécifiques d'un projet via le menu http://localhost:9000/quality_gates.

Complexité cognitive

! INFO

La complexité cognitive dans SonarQube évalue la difficulté de compréhension et de maintenance du code, en tenant compte des boucles imbriquées, conditions et interactions entre les éléments. Lorsqu'une méthode dépasse le seuil fixé, elle devient difficile à lire et à maintenir.

Cette notion est inspirée de la complexité cyclomatique mesurant le nombre de chemins d'exécution possibles dans le code. Une forte complexité cognitive résulte souvent d'une complexité cyclomatique élevée, due à trop de branches, d'imbrications ou de logique complexe.

Ajoutez la classe `Complexity` au projet `quality-demo` pour illustrer la notion de complexité cognitive. La méthode publique de cette classe permet de compter le nombre de voyelles dans un mot donné en paramètre.

Complexity.java

```
1  public class Complexity {
2
3      public String countVowels(String word) {
4          int count = 0;
5          String[] vowels = {"a", "e", "i", "o", "u"};
6
7          for (String c : word.split("")) {
8              for (String v : vowels) {
9                  if (c.equalsIgnoreCase(v)) {
10                     count++;
11                 }
12             }
13         }
14
15         if (count == 0) {
16             if (word.isEmpty()) {
17                 return "empty string does not contain vowels";
18             } else if (word.length() < 5) {
19                 return "word with less than 5 characters does not contain
vowels";
20             } else {
21                 return "word does not contain vowels";
22             }
23         } else {
24             if (count == 1) {
25                 return "contains one vowel";
26             } else if (count < 5) {
27                 return "contains few vowels";
```

```
28         } else {
29             return "contains many vowels";
30         }
31     }
32 }
33
34 }
```

Vérifiez qu'en lançant une analyse, une nouvelle issue liée à la complexité du projet sera détectée.

Ensuite simplifiez cette complexité en corrigeant la classe, par exemple en décomposant la méthode publique :

Complexity.java

```
1  public String countVowels(String word) {
2      int count = countVowelOccurrences(word);
3
4      if (count == 0) {
5          return getNoVowelMessage(word);
6      } else {
7          return getVowelCountMessage(count);
8      }
9  }
10
11 private int countVowelOccurrences(String word) {
12     int count = 0;
13     String[] vowels = {"a", "e", "i", "o", "u"};
14
15     for (String c : word.split("")) {
16         for (String v : vowels) {
17             if (c.equalsIgnoreCase(v)) {
18                 count++;
19             }
20         }
21     }
22     return count;
23 }
24
25 private String getNoVowelMessage(String word) {
26     if (word.isEmpty()) {
27         return "empty string does not contain vowels";
28     } else if (word.length() < 5) {
29         return "word with less than 5 characters does not contain vowels";
30     } else {
```

```
31         return "word does not contain vowels";
32     }
33 }
34
35 private String getVowelCountMessage(int count) {
36     if (count == 1) {
37         return "contains one vowel";
38     } else if (count < 5) {
39         return "contains few vowels";
40     } else {
41         return "contains many vowels";
42     }
43 }
```

Relancez une analyse et vérifiez que l'issue concernant la complexité a disparu. Les issues bloquantes liées à la couverture de test sont traitées dans la prochaine section.

Couverture de test

L'analyse de votre projet échoue toujours car la Quality Gate requiert une couverture de test d'au moins 80%. Pour répondre à cette exigence, ajoutez des tests unitaires en commençant par mettre à jour les dépendances Maven dans le fichier `pom.xml`.

pom.xml

```
1 <dependencies>
2     <dependency>
3         <groupId>org.junit.jupiter</groupId>
4         <artifactId>junit-jupiter</artifactId>
5         <version>5.12.0</version>
6         <scope>test</scope>
7     </dependency>
8 </dependencies>
```

Ensuite créez la classe de test `ComplexityTest.java`.

ComplexityTest.java

```
1 import org.junit.jupiter.api.Test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
```

```
5 class ComplexityTest {
6     @Test
7     public void testEmptyString() {
8         Complexity vowelCounter = new Complexity();
9         String result = vowelCounter.countVowels("");
10        assertEquals("empty string does not contain vowels", result);
11    }
12
13    @Test
14    public void testStringWithoutVowels() {
15        Complexity vowelCounter = new Complexity();
16        String result = vowelCounter.countVowels("bcdfg");
17        assertEquals("word does not contain vowels", result);
18    }
19
20    @Test
21    public void testStringWithOneVowel() {
22        Complexity vowelCounter = new Complexity();
23        String result = vowelCounter.countVowels("cat");
24        assertEquals("contains one vowel", result);
25    }
26
27    @Test
28    public void testStringWithFewVowels() {
29        Complexity vowelCounter = new Complexity();
30        String result = vowelCounter.countVowels("audio");
31        assertEquals("contains few vowels", result);
32    }
33
34    @Test
35    public void testStringWithManyVowels() {
36        Complexity vowelCounter = new Complexity();
37        String result = vowelCounter.countVowels("beautiful");
38        assertEquals("contains many vowels", result);
39    }
40
41    @Test
42    public void testStringWithVowelsIncludingUppercase() {
43        Complexity vowelCounter = new Complexity();
44        String result = vowelCounter.countVowels("Abcdef");
45        assertEquals("contains few vowels", result);
46    }
47
48    @Test
49    public void testStringWithVowelsAndDifferentLength() {
50        Complexity vowelCounter = new Complexity();
51        String result = vowelCounter.countVowels("aeiou");
```

```
52         assertEquals("contains many vowels", result);
53     }
54 }
```

Créez également une classe de test pour la classe `ComplexMethod`.

ComplexMethodTest.java

```
1  import org.junit.jupiter.api.Test;
2
3  import static org.junit.jupiter.api.Assertions.*;
4
5  class ComplexMethodTest {
6      @Test
7      public void testIntToEnglishValue_One() {
8          ComplexMethod complexMethod = new ComplexMethod();
9          String result = complexMethod.intToEnglishValue(1);
10         assertEquals("One", result);
11     }
12
13     @Test
14     public void testIntToEnglishValue_Two() {
15         ComplexMethod complexMethod = new ComplexMethod();
16         String result = complexMethod.intToEnglishValue(2);
17         assertEquals("Two", result);
18     }
19
20     @Test
21     public void testIntToEnglishValue_Three() {
22         ComplexMethod complexMethod = new ComplexMethod();
23         String result = complexMethod.intToEnglishValue(3);
24         assertEquals("Three", result);
25     }
26
27     @Test
28     public void testIntToEnglishValue_Four() {
29         ComplexMethod complexMethod = new ComplexMethod();
30         String result = complexMethod.intToEnglishValue(4);
31         assertEquals("Four", result);
32     }
33
34     @Test
35     public void testIntToEnglishValue_Five() {
36         ComplexMethod complexMethod = new ComplexMethod();
37         String result = complexMethod.intToEnglishValue(5);
```



```
38         assertEquals("Five", result);
39     }
40
41     @Test
42     public void testIntToEnglishValue_Six() {
43         ComplexMethod complexMethod = new ComplexMethod();
44         String result = complexMethod.intToEnglishValue(6);
45         assertEquals("Six", result);
46     }
47
48     @Test
49     public void testIntToEnglishValue_InvalidInput() {
50         ComplexMethod complexMethod = new ComplexMethod();
51         assertEquals(IllegalArgumentException.class, () -> {
52             complexMethod.intToEnglishValue(7);
53         });
54     }
55
56     @Test
57     public void testIntToEnglishValue_Zero() {
58         ComplexMethod complexMethod = new ComplexMethod();
59         assertEquals(IllegalArgumentException.class, () -> {
60             complexMethod.intToEnglishValue(0);
61         });
62     }
63 }
```

Exécutez les tests unitaires via la commande maven suivante afin de vérifier que vos modifications sont prises en compte :

```
mvn clean test
```

PLUGIN SUREFIRE

Surefire est un plugin Maven utilisé pour exécuter les tests unitaires d'un projet. Il fait partie du cycle de vie Maven et est généralement utilisé avec JUnit. Il exécute automatiquement les tests situés dans le dossier `src/test/java`.

Si les tests unitaires ne s'exécutent pas lors de l'exécution de la commande `mvn test`, il se peut que le plugin *maven-surefire-plugin* ne soit pas correctement configuré dans votre environnement. Dans ce cas, ajoutez explicitement le plugin suivant dans le fichier `pom.xml`, dans la section `<plugins>` de la construction du projet :

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
```

Il est important de noter que SonarQube ne calcule pas la couverture de code lui-même. Cette tâche est confiée à un outil externe spécifique au langage du projet. Cet outil génère un rapport qui est ensuite envoyé à SonarQube et intégré dans son analyse. Pour le projet `quality-demo`, il est nécessaire d'utiliser **Java Code Coverage**. La génération du rapport de couverture requiert l'ajout des commandes que **JaCoCo** doit utiliser lors de la phase de **build** du projet. Modifiez le fichier `pom.xml` en y ajoutant ou modifiant les plugins de la section `build` comme ci-dessous :

pom.xml

```
1 <build>
2   <plugins>
3     <plugin>
4       <groupId>org.jacoco</groupId>
5       <artifactId>jacoco-maven-plugin</artifactId>
6       <version>0.8.12</version>
7       <executions>
8         <execution>
9           <id>prepare-agent</id>
10          <goals>
11            <goal>prepare-agent</goal>
12          </goals>
13        </execution>
14
15        <execution>
16          <id>report</id>
17          <goals>
18            <goal>report</goal>
19          </goals>
20        </execution>
21
22      </executions>
23    </plugin>
24  </plugins>
```

```
25 </build>
```

Générez le rapport de couverture de test en utilisant la commande suivante :

```
mvn jacoco:report
```

Vérifiez que le rapport `jacoco.xml` est généré dans le dossier `target/site/jacoco`. Une version consultable est également générée via le fichier `index.html`.

Pour indiquer à SonarQube l'emplacement des résultats des tests unitaires ainsi que du rapport de couverture, ajoutez les informations suivantes dans le fichier `sonar-project.properties` :

sonar-project.properties

```
1 # Répertoires contenant les tests unitaires.
2 # Nécessaire si on veut inclure les résultats des tests.
3 sonar.tests=src/test/java
4
5 # Spécifie l'emplacement des rapports JUnit pour SonarQube.
6 # Nécessaire si on veut inclure les résultats des tests.
7 sonar.junit.reportPaths=target/surefire-reports
8
9 # Rapport de couverture des tests
10 # Obligatoire si on veut suivre la couverture des tests unitaires.
11 sonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
```

Relancez une analyse et vérifiez le succès de celle-ci.

Vulnérabilité

Après avoir exploré l'importance de la couverture des tests dans l'amélioration de la qualité du code, il est crucial de s'intéresser à un autre aspect fondamental : la sécurité du code. Ajoutez la classe `Vulnerability` au projet `quality-demo` pour illustrer la notion de vulnérabilité.

Vulnerability.java

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4 import java.sql.Statement;
```

```
5
6 public class Vulnerability {
7     public void executeQuery(String userInput) throws SQLException {
8         Connection conn =
9         DriverManager.getConnection("jdbc:mysql://localhost:3306/db", "user",
10        "pass");
11         Statement stmt = conn.createStatement();
12         stmt.execute("SELECT * FROM users WHERE name = '" + userInput +
13        "'"); // Injection SQL
14     }
15 }
```

En lançant une analyse, deux nouvelles issues sont créées :

- la première concernant la compromission de votre mot de passe, visible dans le code source
- la seconde concernant une **injection sql** possible

Quelles sont les solutions possibles pour corriger ces vulnérabilités ? Aidez-vous des solutions proposées par SonarQube quand elles sont disponibles.

Analyse d'un projet Python

SonarQube est un outil de qualité de code utilisé pour assurer la qualité de projets logiciels dans divers langages de programmation. Dans cette section vous allez créer et analyser un projet Python avec SonarQube.

Commencez par créer un projet Python appelé `contact_manager` dont voici la structure :

```
contact_manager/
├── src/
│   ├── __init__.py
│   └── contacts.py
├── tests/
│   ├── __init__.py
│   └── test_contacts.py
└── sonar-project.properties
```



ENVIRONNEMENT VIRTUEL

Un environnement virtuel en Python est un espace isolé qui permet d'installer des bibliothèques sans affecter le système global ni les autres projets.

La commande ci-dessous crée cet environnement isolé avec une copie minimale de Python.

```
python -m venv venv
```

Après la création de l'environnement virtuel, il est nécessaire de l'activer avant d'installer des paquets avec *pip*.

Linux/macOS

Windows-cmd

Windows-Powershell

```
source venv/bin/activate
```

Une fois activé, vous verrez généralement le nom de l'environnement (*venv*) apparaître au début de la ligne de commande, indiquant que vous travaillez bien dans l'environnement isolé.

pip est le gestionnaire de paquets officiel pour Python, utilisé pour installer, mettre à jour et gérer des bibliothèques ou modules Python.

Vérifiez que *pip* est disponible dans votre environnement avant de passer à la suite :

```
pip --version
```

Dans le dossier `src` créez le fichier `contacts.py` avec le code ci-dessous :

contacts.py

```
1 class Contact:
2     def __init__(self, first_name, last_name, phone, email):
3         self.first_name = first_name
4         self.last_name = last_name
5         self.phone = phone
6         self.email = email
7
8     def __str__(self):
9         return f'{self.first_name} {self.last_name}'
10
11 class ContactManager:
12     def __init__(self):
```

```
13         self.contacts = []
14
15     def add_contact(self, contact):
16         self.contacts.append(contact)
17
18     def remove_contact(self, contact):
19         if contact in self.contacts:
20             self.contacts.remove(contact)
21
22     def find_contact(self, first_name, last_name):
23         for contact in self.contacts:
24             if contact.first_name == first_name and contact.last_name ==
last_name:
25             return contact
26         return None
```

Dans le dossier `tests` contenant les tests unitaires créez le fichier `test_contacts.py` suivant :

test_contacts.py

```
1  from unittest import TestCase
2
3  from src.contacts import Contact, ContactManager
4
5  class TestContactManager(TestCase):
6
7      def setUp(self):
8          self.manager = ContactManager()
9          self.contact1 = Contact("John", "Doe", "1234567890",
"john.doe@example.com")
10         self.contact2 = Contact("Jane", "Smith", "0987654321",
"jane.smith@example.com")
11
12     def test_add_contact(self):
13         self.manager.add_contact(self.contact1)
14         self.assertIn(self.contact1, self.manager.contacts)
15
16     def test_remove_contact(self):
17         self.manager.add_contact(self.contact1)
18         self.manager.remove_contact(self.contact1)
19         self.assertNotIn(self.contact1, self.manager.contacts)
20
21     def test_find_contact(self):
22         self.manager.add_contact(self.contact1)
23         result = self.manager.find_contact("John", "Doe")
```

```
24         self.assertEqual(result, self.contact1)
25
26     def test_find_contact_not_found(self):
27         result = self.manager.find_contact("Non", "Existant")
28         self.assertIsNone(result)
```

À la racine de votre projet, créez un fichier `sonar-project.properties` pour configurer SonarQube.

```
# Identifiant du projet.
# Doit être unique dans l'instance SonarQube.
# Obligatoire
sonar.projectKey=contact-manager-python

# Nom du projet tel qu'il apparaîtra dans l'interface web.
# Obligatoire
sonar.projectName=Application de demonstration en Python

# Permet de suivre l'évolution de la qualité entre différentes versions
# Facultatif
sonar.projectVersion=1.0

# Répertoire de base du projet
# Par défaut SonarScanner utilise le dossier où il est exécuté.
# Facultatif
sonar.projectBaseDir=.

# Version de python
sonar.python.version=3.8

# Spécifie où se trouvent les fichiers source à analyser.
# Obligatoire
sonar.sources=./src

# Répertoires contenant les tests unitaires.
# Nécessaire si on veut inclure les résultats des tests.
sonar.tests=./tests

# Rapport de couverture des tests
# Obligatoire si on veut suivre la couverture des tests unitaires.
sonar.python.coverage.reportPaths=./coverage.xml
```

La clé `sonar.python.coverage.reportPaths` du fichier de configuration a besoin du rapport de couverture de tests. Pour obtenir ce rapport, installez la librairie `coverage` : `pip install`

`coverage`. Calculez ensuite cette couverture de tests via la commande `coverage run -m unittest discover -s tests` et générez le rapport final grâce à la commande `coverage xml -o coverage.xml`

Lancez l'analyse de ce projet et consultez le rapport dans SonarQube afin de vérifier que ce rapport a été pris en compte lors de l'analyse.

Analyse de vos projets personnels

Sélectionnez un projet que vous avez développé dans un cours de développement ou de web. Configurez SonarQube pour analyser ce projet et essayez de faire passer la Quality Gate à celui-ci.

Pensez à utiliser SonarQube pour vos futurs projets.